

Lec 14 Regularization Techniques in AE: Sparse

Lec 14 Regularization Techniques in AE: Sparse

Lecture	14 / 63 + intro
Duration	39:30
Captions	English (auto-generated)
Words	6,009
Video	CmOGqsDRVP8
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone, welcome to the lecture series on fundamentals of generative VI and large language models. We in the last session we had uh seen the uh denoising autoencoder. In this particular session we will see the next regularization technique which is a sparse autoencoder. So what exactly um from our previous session itself you would have understood that it is mainly used to avoid the um overfitting problem. Suppose if you have a data set uh in this particular uh session um in this uh uh slide what exactly we are seeing. Suppose we have a data set where the number of dimensions of um hidden representation is less than the dimension of an input. So when we have an architecture of this nature so is it

[00:50] not uh you are forcing the model to compress the input data right the model is forced to compress the input data. Hence the model is definitely a little attentive when learning the important uh uh features. [snorts] Now um uh second what if you have an uh dimension of an input greater than the um size of an uh or the dimension of an hidden repres uh the dimension of an hidden representation. What if it is greater than the input? Suppose if you have uh um something like this, if you have uh in this way u uh so what exactly is happening is it not uh there is a very high chance that the

model may um you know simply copy the uh input without learning the meaningful features. So we definitely require an regularized one.

[01:43] So this is what is the introduction we have already seen and even the diagram also remained the same as our previous one. Here the dimension of an hidden representation is much larger comparing to an input. Now uh how about this requirement? Even if the dimension is large uh still you should uh uh the requirement here is even if the dimension is large still you should enforce the model uh to learn the uh features. Uh there shouldn't be an uh mapping uh s simple identical mapping.

[02:17] It should not be identity mapping. it should not be. [snorts] So even if the hidden representation uh number of neurons in the hidden layer are more you should still uh make the model to um learn the important features. How it is possible? Even if there are high can I um you know make this output as zero this output as a zero and this as a sparse zero. Can we do this? In that case what has happened number of neurons are now less. So with this is it not again making sure that it is learning the most important features. Are we uh are you getting this concept? Even if the hidden uh uh layer has many neurons, our requirement is we still want the model to learn the important features.

[03:04] How? Now we will be applying the sparsity to the hidden activation. What is uh uh uh sparsity here? We'll understand that uh uh in a while. But the whole requirement here is we'll be introducing the sparsity uh to the hidden activation. Now with this let us see what exactly is the um sparse autoencoder. So sparse autoencoder is most hidden neurons. We want the most hidden neurons to stay inactive for most input or um you know make only few hero neurons to be active. When there are only few neurons which are active then we are enforcing it to capture the most important features of the input. So are we getting this? Uh the whole idea here is we have more number of neurons because more number of neurons the

[04:00] problem is overfitting poor generalization. We don't want that. So we want a regularization technique. What is that regularization technique uh it is doing here is even though there are more [snorts] keep only few neurons active most of the neurons should be inactive. So by doing this is it not um we are um restricting the model when there is a restriction when the neuron is active it should capture the important features. Now uh this is what is a sparse autoencoder. If this is the case we should the whole concept here is you should add that sparsity. How is that you'll be adding the sparsity? By adding a sparsity penalty to the loss function.

[04:48] So, so far in a plain autoencoder you just had only the reconstruction uh uh loss term. Now to that uh uh reconstruction loss term you are adding one more additional uh term that one more additional term is sparse penalty. Now in this uh we know that uh L reconstruction is how well the input is reconstructed and L sparsity is penalty when there are too many hidden neurons become active because

we don't want if the neuron is active punish it give the penalty so that it becomes less active when it becomes less active the remaining features um remaining neurons will capture the important uh uh features So I think uh you are getting the overall idea. So we don't want a neuron to be active for most number of times.

[05:44] So it should be remain inactive. So when you make that neuron active then capture the uh important um you know u features from the data or important information from the data from the input. And we have a beta which is mainly used to control the sparsity. how much sparsity should be introduced. So that beta is a um hyperparameter here. So it's a small constant that will be added. So [snorts] uh now when I say sparsity, this sparsity can be added in two ways. One either you can go for KL divergence or we can go for L1 regularization. So in this particular uh session and the original research paper also has applied KL divergence. So we'll be learning that with the KL divergence. Now what are we understanding? Ultimately hidden neuron

[06:40] should be restricted. Right? If that is the case, look at the diagram that is mentioned here. Now I'm passing in the uh data set I have um many number of samples and there are four features. Those four features are what is been drawn here uh given here. Now there are three hidden neurons. uh just for an example I'm giving you three uh it can be any number of neurons it's okay whether it's undercomplete over complete any this um for overcomplete for sure there is an problem so that doesn't mean that in undercomplete we will not have so whatever [snorts] may be the architecture how do we avoid this uh uh problem let us look into so we have an uh uh data set which is having four features and there are four neurons now

[07:26] when I pass the first input so uh uh so when I pass the first input you have a connection like this. You will have uh a connection in this way and uh uh you will have even connection like this. Uh let me uh erase this. So you will have connection to each of this. Then you will have connection to each of this. Right? This is what is a fully connected one. Now uh what is a sparse autoencoder is uh uh saying the hidden neuron should remain inactive. Uh that's what it is saying. So which means the focus is more towards this hidden uh uh neuron. Now so here uh what exactly is this uh uh notation? You can see all of a sudden we are seeing a different notation because I'm trying to retain the same notations as that of the

[08:28] original paper. So here what exactly uh the notation is X here is an input X and two indicates at a layer two first layer and then the second layer two and this J indicates this neuron J and A indicates the output activation that is um output of that hidden neuron is A. Uh so uh uh if there is one sample there is one sample that you are passing or the first uh the first uh sample or one sample X is given to a network then the activation of the hidden this is summation this is activation the activation of hidden neuron J is denoted as this. Now what is this whole term?

[09:16] How is that? You will be reading this A is output of hidden neuron J in a layer

two for the input X . This is how you'll be reading it. Right? Okay. So now in general instead of passing a specific input I'm telling in general I training sample. How is that? You'll be writing E training sample. The E training sample is denoted in this way. Okay. Now let us understand uh can we is this uh uh approach is good that is for one input um you are making that hidden neuron uh inactive for one more input the hidden uh neuron inactive or can we take up the average of all the samples for example what I'm trying to say is when you pass the first sample here you will get one value when you pass the second sample

[10:10] you will get the second value when you pass the third uh uh input you will get uh the third value fourth uh uh uh sample we'll get the fourth value here fifth and you'll get the fifth value so how many ever training samples you are passing you will have so many outputs for each of the neuron correct so can we now take the average for all of these so can you look at this average over the whole training u uh set so how is that you will take the Average this is for a specific I 1 average over m samples is add up all of this $1 + 2 + 3 + 4 + 5$ all the values which are coming at that particular neuron add up all of this and then take the average 1 divided by m or whole divided by m is number of such

[11:02] samples what did we do here so um I have just explained you independent individually each of these terms but what did we do so far. So if a training samples has training a data set is having m samples here we are going to have m values agree not only there here also we will have m values here also we will have m values how m values when you pass the first sample is it not the connections are made in this way uh uh so when you pass the second again second data when you pass here the connections are like this so you will get the second value like this when there are m values m samples you pass m such values we get here okay so take the average of all of those that's all you understand at this

[11:54] point of time now let us understand why uh we have taken this average what is the whole intention so what is a sparse autoencoder now within uh uh that hidden layer now this is the hidden layer here within this hidden layer layer only a small fraction of a neurons should be active. So now within this there are three neurons. So out of these three a small portion of the neurons should be active at a time or either you consider that or on average each neuron should be active only rarely. Uh you got the concept. So within the hidden layer if at all if you have a three keep only a few neurons active either do that way or because you have considered an average what is average is saying this is uh there is one average here there is

[12:53] one average there is one average on an average each neuron should be active only uh for some time or only rarely it should be active. So which means if this neuron is active very rarely when it is activated is it not its duty is to capture the uh important features. This neuron will be very rarely active when it is active it has to capture the important uh information. Are you getting the intuition behind this? So either make

few neurons active or on an average each neuron should be active very rarely. Okay. So now let us uh understand this uh little more mathematically. [snorts] Now there are two things that we there is one thing that we have seen what is that you have taken an average by taking an average

[13:46] that what is an average of all the values which are there at the first neuron you are taking the average of it you'll get one value that value we call it as $\hat{r}$. Okay. Now what is that activation we have got? I mean that value we got it is an actual average activation. Now if it is an actual average activation [laughter] we should have one desired average activation or a sparsity penalty one. So that we call it as r . So now we have two terms r and $\hat{r}$. Okay r is a desired and $\hat{r}$ is actual. So what is that sparse penalty or activation uh um or auto sparse autoencoder? What is the sparse autoencoder requirement? The sparse autoencoder requirement is whatever is the average that I have obtained that

[14:39] should be as close as the desired one. So this is the constraint. The constraint here is the desired activation function should be equivalent to or should be as close as the actual one. Okay, this is the constraint. Now when it is very close to uh when uh this value when both are very close then give less penalty. When they are very far from each other then give more penalty. Right? So when two now you have just two values. What are those two values? r and $\hat{r}$. If r and $\hat{r}$ values are very close to each other, which means add less penalty. If r and $\hat{r}$ are very far from each other, then give more penalty. Uh uh so far you have understood this. Now if at all if you want to know how close they are, how

[15:35] far they are, which means you have to measure the distance between these two. You require some some metric to measure. So how much penalty to give how far they are how close they are that is measured using a KL divergence right. So what is a KL divergence here KL divergence is measuring the you know distance between these two how much P and r and $\hat{r}$ are you know differing or how close they are or how far they are. So KL divergence is used to measure the penalty. How much penalty to give. This is what is the complete formula for KL divergence. So in KL divergence we have the first term and we have the second term. What is that? We will understand shortly. the uh if P and um I mean if

[16:30] r and $\hat{r}$ if they are very close to each other which means they are almost equal then the KL divergence value will be zero. When KL divergence value is zero which means there is no penalty or there is a very small penalty that is given because ideally in a real world never these two will be equal it will be always um approximate very closely. So either there is a small penalty or there is zero penalty. When this value and this value are very far from each other then definitely this value increases. When this value increases there is a large penalty that is given. So um I have still not explained you about this formula. So far what we have seen is our requirement is um you have to avoid the overfitting

[17:20] problem. So what is that we are doing? Even if there are more number of neurons at a hidden layer, the requirement here is the neuron should be on an average every neuron should be active very rarely. If every neuron should on an average if every neuron should act rarely uh should be active uh rarely then you should compare that uh average with some um ground truth average that ground truth or decide is row and the actual average what we have got is row hat. Now you should compare how far they both are. So for that comparison we require Kullback-Leibler divergence and how is Kullback-Leibler divergence formula. This is what is the formula that is given. Now let us understand. Um it is just not for one uh neuron. We have all the in the hidden

[18:15] layer we have many such neurons. You will be adding up all of those. So how is that you will add? If there are um four such neurons it is the KL divergence between the first one, second, third and fourth. You will add up all of those penalties. Now let us understand the slide is important because you will get the actual intuition here only. So here there are two parts. One is active part. The first term in the KL divergence we call it as an active part. Second uh uh term we call it as an inactive part. Now what exactly is this active and inactive? What is our requirement? Our requirement is there is a neuron at a hidden layer and the uh the neuron behavior can be either active or it can be inactive. That's what is our

[19:03] requirement. Most of the time it should be inactive but sometime it should be active also which means one particular neuron can act either as can be active or inactive. So its neurons behavior has two sides active and inactive. Can we compare only how far the neuron can we check only the active side? That's not correct, right? So we will not fully capture the neuron's behavior. If you check only the active side, therefore we will be using a KL uh divergence formula. What is KL divergence formula here? It will be using it one term is an active term, one term is an inactive. we will add up them together so that um the total mismatch between these two becomes a penalty that's what whether to give

[19:56] more penalty less penalty you cannot just consider only one term you should consider both the terms this is a theoretical explanation let us see this mathematically or with one numerical example let us take the uh we have row and we have row hat so row is something which is desired one. Let us assume that the desired is 0.05. So what is that? We are saying the desired uh average activation should be 0.05. So here there are two. So desired active part should be 0.05 and desired inactive. So how is that you will be see can you see in the formula you have this term and you have this term also. So the uh this row says active one minus row says inactive right. So if row is 0.05 that is desired active part then desired inactive part

[20:58] which is 1 minus row is 0.95. Uh can you able to understand what is that we have uh calculated? We have just calculated given desired from this what is desired inactive we have calculated. Now look at uh uh this entire complete problem. Now if this is the desired one then what is the actual uh um uh you know average activation

the actual average activation is for example 0.05 which means I'm giving a condition where row is exactly equivalent to row hat so row value is 0.05 and row hat actual row um is 0.05 05 again in this you find out what is uh actual active and actual inactive. So the actual active is 0.05 and actual inactive which we are calculating this part is 1 minus 0.05 which is 0.95.

[21:56] Uh have you got uh this concept so far? Now just substitute the values in the KL divergence one. What is the uh this is 0.05 and this is also 0.05. That's what is both the terms. Then this value is 0.05 log. Uh here I have considered log to base 10. So this is 0.05 and this actually is also 0.05. Then 1 minus row is 0.95. So this is 0.95 and this is 0.95. So can you see this? Uh so since log 1 is 0. Uh so the whole term what we are getting is zero which means uh um here the row and row hat are same uh in our previous slide have we we have seen right when this is the k divergence term is zero that's what I'm giving one example to you here so here um the k divergence what we got is zero now let

[22:55] us understand what is this active part inactive part now the first term here the First term what we have what it is going to check is it is going to check uh the correctness of active part what do you mean by this the uh actual one is 0 I mean the actually desired is 0.05 and the actual active is 0.05 whether that part um is it correctly matching so since here 0.05 and 0.05 05. They are perfectly matching. There is a perfect match. What this is saying, how often the neuron should be active versus how often the neuron is actually active is what the first part has checked. Second, what is that? The second it has checked this desired inactive is 0.95 and actual inactive is 0.05. That is also matching because there is a perfect

[23:54] match. the total when you add it is zero. So what is this uh term saying? This term says how often the neuron should be inactive versus how often the neuron is actually inactive. Did you get this concept? So here um KL divergence it is considering both that is um you know how often the neuron should be active versus actual active and how often the neuron should be inactive versus actual inactive. This will give us the overall behavior of a neuron. So that is captured using a KL divergence which has two terms. The one is checking an active part. Second is checking an inactive part. I hope this is clear to you. Now let us get into an sparse autoencoders with the complete numerical example so that you will

[24:48] understand what is this sparse autoencoders is all about. Let us assume there are five samples. When I say five samples, what is the meaning of it? There is an data set and the data set is having five features. There are five features and there is a first sample, second sample, third sample, fourth sample and fifth sample. There are five such a samples. When you pass the first sample, when you pass the first sample to this, is it uh not you get the value here, here, here, here, you'll get the values for all the seven.

[25:27] Do you agree to this? When you pass the first one, you will have the connection uh first input, you'll have connection to all of this. You'll get this value. Say for the same sample when um you will have a connection to all of this. Next you will have connection to each of this then you will have connection uh to each of this and then you will be getting so totally you have obtained the values. So what is that for uh for the first uh uh sample we have for the first sample we have these many then uh uh you will take the second sample when you pass the second sample you'll get the value here here in all these places you will get the value that value is this so on so forth all the training samples you have

[26:15] passed and you have obtained so if there are five such samples do you agree you will have five such values Here you will have five such values. I think uh uh you all agree to this. There are five such values. Now those five such values are each column. Correct? These are five such values. Five such values. Do you agree to all of this? Answer is yes. Now what is that we are? Because there are five such values. you should take an average of it. How is that you will take an average? This is the formula to take an average. The um you know activation you will add up all of this and you will divide it with the number of such a samples. So you will add and you will divide it with five for this particular

[27:08] example. Now what you got the uh average of this is 0.3 um it is 0.3. So this is the value that we have obtained. Similar way you will calculate this is the average activation of hidden neuron one then you will calculate like this for all other uh uh neurons. So for uh H1 the average u um activation is this much for H2 this is the value all these values what did we do simple we have taken an M training samples training and then we have passed it to the model we have obtained m such values for each of the neuron we have considered the we have calculated the average and the average is shown in this table. Now uh what we have obtained is row hat. There is a desired one. Let us assume that the desired um sparse uh is

[28:08] 0.005. Why is that 0.5? This is just one example. So here the actual average activation is 0.005. Now we have to calculate the KL divergence. How far they are? How close they are? Just by looking at this from human eye. When you look at this is it not this value is very much close to this value. This is very far close close. This is very far comparing to this value and this is very uh close. So now when uh can you able to uh write this what is that we have calculated? We have calculated row hat. This row hat you are comparing it with the actual row. So this value is very close to this value. Okay, this is very very far.

[29:01] This is very close, right? Very very close. And this is okay. Somehow it is close only. This is close. This is very far. This is close. So now the whole concept here is we don't want a hidden neuron to be active for most of the time. the average activation of that particular neuron should be active only rarely. If that is the case uh you have to check the average activation. We have done that. You have compared

it with the original. We have done that. Now where the penalty should be more definitely the penalty should be given to H2 and HS6 uh H2 and HS penalty should be given. Why penalty should be given?

[29:49] Because on an average it is these two neurons which are active for most times. So we don't want uh sparse autoencoder doesn't want this. So neuron H2 and HS6 are much more active. So therefore they are firing too often. Sparse autoencoder does not want this. So if it does not want this it has to give the penalty. How is that it will give the penalty? Now see this is the table what I have taken it from the previous one.

[30:18] We have an desired uh sparse which is 0.05 uh uh 0.005. Now you have to calculate the penalty. How is that you will calculate the penalty? You have both the values. You have row and row hat. So with this just substitute the values into it and then when we can calculate what is uh uh the k divergence uh value. [snorts] So I'm just showing you for one example and the remaining I'm writing it directly. So here uh row value be 0.05 uh uh 0.5.

[30:52] So that is a row and 0.3 is um uh row hat. What is row hat? It is an average uh activation of that particular neuron. So just substitute the values you are going to get this calcul this value. So once again wherever there are numerical values you can calculate by yourself and if there are any typos you can correct it. So here you have got this penalty. Now whatever is this this is the penalty. Okay. Now uh uh can you correlate uh the average value 0.3 is very much closely matching with 0.005 right? So when it is a close match the penalty should be very minimum or it should be zero. So here you see there is a very small penalty that you are adding. So similar way do it for all other values. Second is the actually 0.5

[31:49] and then the uh desired value is 0.0 uh um I mean 284. So calculate the values substitute the values into this formula and then you get this value. Similar way you get this value. So from this table what are we understanding? Clearly we are understanding that H2 and H6 um is the neuron that is average um uh you know activation by considering the average activation we are seeing H2 and H6 is active for most number of uh uh times.

[32:24] So we don't want kale divergence doesn't want that. So if that is the case more penalty should be given for H2 and H6 when more penalty is given for H2 H6 is it not it is mathematically proved here the penalty is high for these two uh neurons. So when penalty is high for these two definitely during a back propagation there will be associated weights for these hidden neuron those weights will be adjusted. When those weights get adjusted, what is happening?

[32:56] Is it not you are restricting H2 and H6? Now H2 and H6 will not be active for most of the time. So here you are imposing the restriction for hidden uh neurons. So here clearly the main penalties are coming from H2 and H6 and KL divergence gives

a larger penalty to H2 and H6. So um this is what is the uh concept. Now on average neuron is going uh uh to be an inactive. What does it mean? It means whenever it is active um u then it has to really going to capture most relevant uh information.

[33:39] This is what is the hidden intuition right? It is uh inactive for most of the time but when it becomes active it has to capture the information. Is it not some sort of a constraint we have put? It's not very free the way that it was uh uh in autoencoder. There is some sort of a restriction we have put some penalty because of the penalty. Most of the time it is inactive but when it comes active it has to capture. So this is how you are uh going to impose uh um you know you are achieving the regularization.

[34:09] So this is the final uh uh slide for sparse autoencoders where the overall objective here is reconstruction and for this you are adding the penalty and that penalty here is the um you know KL divergence one sparse penalty sparse penalty here is KL divergence. Now you see here uh this particular diagram will give us the entire intuition you have passed an input. Now in this input say for example uh the uh row is 0.5 and the row hat is 0.3 and this is let us assume uh 0246 and this is 04.

[34:53] I'm just giving one example to summarize this. Now when you look into this on an average H2 is something which is active most of the time but uh sparse autoencoder says you should not allow it to happen if it is uh if it has to restrict this just imagine if the uh average activation if it is more which means this is the neuron which is active. So what sparse uh autoencoder ensures give more penalty. How it gives more penalty? KL divergence. KL divergence automatically increases. Why it increases? When it compares this with this definitely the value increases.

[35:37] When the value increases is it not the um penalty increases? Now what exactly happens uh during the back propagation? Uh now why penalty increases? This term is definitely high. When this term is added to the loss definitely the overall loss value is more the total loss value is more. Now because the loss is more during back propagation when you back propagate it for uh uh H2 are there not these are the values which are weights which are associated with H2. Right? So during a back propagation the values which are associated with the neuron that is hyperactive or overactive what it will do it will come and adjust the weights. When it adjust the weights definitely for the next time H2 will become less active. So when uh it

[36:33] becomes less active is it not you have imposed certain restriction for the hidden neuron. So this is all about the uh sparse autoencoders. During a back propagation the network adjusts the weights and bias so that the activation of that neuron decreases. In this way the overactive neurons are pushed to become less active and they are um uh less fired neurons. So because the network is not allowed to use um most of the hidden neurons freely, it can no longer have that overfitting problem, right? Instead it is forced to learn u the uh it you know you are enforcing that uh to use only few active neurons so that when the neurons are active less number

of neurons active it is going to capture the most important information in

[37:32] the data. So this is what is the sparse autoencoder. So overall it is you are imposing a certain restriction to the hidden neuron uh hidden neurons. How you are imposing the restriction? Your requirement is on an average the neuron should be active rarely. Now how are you ensuring that if it is active often you are punishing it. How are you punishing it? KL divergence. You are comparing the average with the desired one. If the uh uh values are too different, the KL divergence term increases. When KL divergence term increases, it is being added to this uh to the loss. Right? When this term increases, automatically uh when it is added something to this, the overall loss increases. When the loss increases,

[38:22] when you're back propagating, you have to update the weights. Now, how are you updating the weights? So definitely these weights of course all the uh weights gets updated but uh wherever the neuron the penalty is more that particular weights gets adjusted and its bias also gets uh adjusted. So with this you are making neuron less uh active. So with this you have introduced some sort of a constraint. uh when there is a constraint, it cannot just relax by copying the input uh to an output. It cannot because you're not allowing it to be active. So when it is active, it has to capture. So that constraint uh leads to a regularization. So with this we have come to an end of this particular uh uh session. In the next session we

[39:17] will see the last type of regularization which is a contractive autoencoders. So thank you.